МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа № 3**
**по курсу «Операционные системы»**

**Вариант 12**
**Многопоточная эрозия и наращивание**
**матрицы вещественных чисел**

Выполнил: Г. П. Графов
Группа: М8О-203БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

## Условие

**Цель работы.** Приобретение практических навыков управления потоками операционной системы и обеспечения синхронизации между ними.

**Общее задание.** Разработать программу, обрабатывающую данные в многопоточном режиме с использованием стандартных средств создания потоков ОС. Максимальное число одновременно работающих потоков задаётся параметром запуска. Продемонстрировать фактическое количество потоков средствами операционной системы. Исследовать зависимость ускорения и эффективности от размера входных данных и количества потоков.

**Вариант 12.** Выполнить $K$ последовательных применений фильтра эрозии и $K$ последовательных применений фильтра наращивания к матрице вещественных чисел. Получить две результирующие матрицы.

**Принятые соглашения реализации.**

- Используется C++17. Работа с потоками выполняется через интерфейс `mt`; в Linux-реализации библиотеки используются функции POSIX Threads. `std::thread`, `std::mutex` и `std::filesystem` не применяются.
- Фильтры независимы: обе цепочки преобразований начинаются с исходной матрицы, а не с результата другого фильтра.
- Структурирующий элемент — квадратное окно нечётного размера $W \times W$. На границе рассматривается только пересечение окна с матрицей.
- Входные значения имеют тип `double` и должны быть конечными. Размеры матрицы положительны, $K \geq 0$, запрошенное число потоков $P \geq 1$.
- Ограничение $P$ включает главный поток. При $K > 0$ число участников равно $p = \min(P, N)$, где $N$ — число строк. При $K = 0$ дополнительные потоки не создаются, а обе выходные матрицы равны исходной.

## Входные и выходные данные

Первая строка входного текстового файла содержит число строк и столбцов $N, M$. Далее записываются $NM$ вещественных значений. Лишние токены, недостающие элементы, `NaN` и бесконечности считаются ошибкой.

Программа сохраняет результаты в файлы с суффиксами `_erosion.txt` и `_dilation.txt`. Формат каждого результата совпадает с форматом входа: размеры и элементы матрицы.

## Метод решения

## Эрозия и наращивание

Пусть $A$ — исходная матрица размера $N \times M$, а $r = \lfloor W/2 \rfloor$ — радиус окна. Для позиции $(i, j)$ множество допустимых соседей определяется как

$$\Omega_{ij} = \{(x, y) : 0 \le x < N,\ 0 \le y < M,\ |x - i| \le r,\ |y - j| \le r\}.$$

Один проход эрозии и наращивания вычисляется по формулам

$$E^{(0)} = A, \qquad D^{(0)} = A,$$

$$E_{ij}^{(s+1)} = \min_{(x,y)\in\Omega_{ij}} E_{xy}^{(s)}, \qquad D_{ij}^{(s+1)} = \max_{(x,y)\in\Omega_{ij}} D_{xy}^{(s)}.$$

После $K$ проходов возвращаются $E^{(K)}$ и $D^{(K)}$. При $W = 1$ каждый элемент является единственным элементом своего окна, поэтому обе операции сохраняют матрицу при любом числе проходов.

## Распределение работы

Матрица разбивается по строкам между $p = \min(P, N)$ участниками. Участнику с номером $q$, $0 \le q < p$, назначается полуинтервал

$$\left[\left\lfloor \frac{Nq}{p} \right\rfloor, \left\lfloor \frac{N(q+1)}{p} \right\rfloor\right).$$

Диапазоны не пересекаются, покрывают все строки и отличаются по длине не более чем на одну строку. Участник с номером 0 выполняется в главном потоке; для остальных создаются $p - 1$ дополнительных потоков. Каждый участник обрабатывает свой диапазон сразу для двух независимых фильтров. Отдельные группы потоков для минимума и максимума не создаются.

## Буферы и синхронизация

Для каждого фильтра выделяются два буфера. На проходе $s$ читается буфер $s \bmod 2$, а результат записывается в буфер $1 - (s \bmod 2)$. После прохода каждый участник ожидает остальные на барьере. Поэтому никто не начинает чтение следующего поколения, пока оно не вычислено целиком. По окончании возвращается буфер с индексом $K \bmod 2$.

Гонки при записи элементов отсутствуют: разные участники записывают разные строки. Входной буфер текущего прохода только читается. Мьютекс не нужен для каждого элемента матрицы; он применяется для согласованного старта и неделимого вывода диагностических строк.

## Безопасный запуск потоков

Перед созданием дополнительных потоков главный поток блокирует стартовый мьютекс. Созданные участники ожидают его освобождения. Если все потоки созданы, блокировка снимается и запускаются вычисления. Если создание одного из потоков завершилось

ошибкой, устанавливается признак отмены старта. После снятия блокировки уже созданные участники возвращаются, не входя в барьер. Главный поток ожидает их завершения и сообщает об ошибке. Это исключает ожидание на барьере участника, который не был создан.

## Оценка сложности

При прямом переборе окна трудоёмкость двух фильтров имеет порядок $O(KNMW^2)$. Если окно превышает размеры матрицы, фактическое число посещаемых соседей ограничено её размерами. В идеальной модели вычислительная часть делится между $p$ потоками, но к ней добавляются создание потоков, синхронизация и ожидание планировщика.

Основные буферы содержат четыре матрицы `double`, что требует примерно $4NM$ элементов; размер одного элемента равен `sizeof(double)` байт. Также выделяются задачи и дескрипторы потоков: $O(p)$. В эту оценку не входят стеки системных потоков и служебная память библиотек.

## Описание программы

## Организация файлов

| Файл | Назначение |
|---|---|
| `common.hpp` | Параметры запуска, структура матрицы, общие функции. |
| `filters.hpp` | Структура результата и интерфейс `run_filters`. |
| `main.cpp` | Разбор аргументов, чтение, запуск обработки, запись результатов. |
| `common.cpp` | Преобразование целых чисел, диагностика, PID/TID и таймер. |
| `matrix.cpp` | Выделение памяти, копирование, проверка и файловый ввод-вывод. |
| `filters.cpp` | Вычислительные задачи, двойная буферизация, запуск через `mt`. |
| `multithread/mt.h` | Общий интерфейс создания и ожидания потоков. |
| `multithread/mt_posix.cpp` | Linux-реализация функций интерфейса `mt`. |
| `mt_sync.h`, `mt_sync_posix.cpp` | Обёртки над мьютексом и барьером POSIX. |

## Основные структуры

`Options` хранит путь входного файла, префикс выходных файлов, ограничение числа потоков, число проходов, размер окна и признак подробного вывода. `Matrix` содержит размеры и непрерывный массив `double`, управляемый `std::unique_ptr`. Матрица перемещается, но не копируется неявно; для явного копирования используется `copy_matrix`.

`Context` содержит буферы двух фильтров, параметры вычислений, барьер, мьютексы и признак отмены старта. `Task` задаёт контекст, номер участника и границы его строк. `Result` возвращает две матрицы, время вычислений и фактическое число участников.

## Интерфейс библиотеки потоков

Основные операции интерфейса:

- `mt::CreateThread` — создание потока;
- `mt::ThreadJoin` — ожидание завершения;
- `mt::DestroyThreadHandle` — освобождение дескриптора;
- функции `mt::MutexCreate`, `mt::MutexLock`, `mt::MutexUnlock` и `mt::MutexDestroy` — работа с мьютексом;
- функции `mt::BarrierCreate`, `mt::BarrierWait` и `mt::BarrierDestroy` — работа с барьером.

Платформенные операции скрыты в Linux-части библиотеки. Сигнатуры создания и ожидания соответствуют интерфейсу проекта репозитория преподавателя. Текущая реализация лабораторной рассчитана на Linux/WSL; наличие пространства имён `mt` само по себе не означает готовность всей лабораторной к нативной Windows.

## Функции ОС и библиотек

- `pthread_create`, `pthread_join` — создание и ожидание системных потоков внутри Linux-реализации `mt`.
- `pthread_mutex_*`, `pthread_barrier_*` — взаимное исключение и синхронизация проходов через обёртки библиотеки.
- `clock_gettime(CLOCK_MONOTONIC)` — измерение интервала вычислений; системное календарное время для этого не используется.
- `getpid`, `syscall(SYS_gettid)` — идентификаторы процесса и Linux-потока для диагностики.
- `sigaction`, `sigemptyset`, `write` — обработка `SIGPIPE` и диагностический вывод с повтором при `EINTR`.
- `getopt`, `strtol`, `strtod`, `fopen`, `fscanf`, `fprintf`, `fclose` — функции библиотек для аргументов и файлового ввода-вывода.

Функции `pthread_*` являются библиотечным интерфейсом, а не одноимёнными системными вызовами ядра. Создание потоков в трассировке Linux видно как `clone` или `clone3`; ожидания при конкуренции могут использовать `futex`. Семантика описана в pthread_create(3), pthread_join(3) и pthread_barrier_wait(3p).

## Параметры запуска

| Ключ | Назначение |
|---|---|

| | |
|---|---|
| `-i FILE` | Обязательный путь входной матрицы. |
| `-t P` | Максимальное число потоков, включая главный; по умолчанию 1. |
| `-k K` | Число проходов каждого фильтра; по умолчанию 1. |
| `-w W` | Нечётная положительная сторона окна; по умолчанию 3. |
| `-o PREFIX` | Префикс результатов; по умолчанию `result`. |
| `-v` | Вывод PID, TID и диапазонов строк участников. |
| `-h` | Краткая справка по запуску. |

## Результаты

## Условия проверки

Замеры производительности выполнены 12.11.2026 на компьютере студента с процессором Intel Core i7-13650HX в WSL2. Система: Linux 6.6.87.2-microsoft-standard-WSL2, x86-64, glibc 2.39. Среде доступны 20 логических CPU. Использована текущая версия лабораторной с интерфейсом `mt`, собранная CMake в режиме Release. Версия компилятора в журнале окружения не зафиксирована. Исходные результаты: `inc/measurements/raw.csv`; итоговая таблица: `inc/measurements/summary.csv`; сведения о системе: `inc/measurements/environment.txt`.

Следующие проверки небольшой матрицы и количества потоков относятся к ранее подготовленной контрольной Linux-сборке. Они приведены отдельно и не являются журналами новых замеров на компьютере студента.

## Проверка на небольшой матрице

Для исходной матрицы

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

при $K = 1$, $W = 3$ получены

$$E = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 1 & 2 \\ 4 & 4 & 5 \end{pmatrix}, \qquad D = \begin{pmatrix} 5 & 6 & 6 \\ 8 & 9 & 9 \\ 8 & 9 & 9 \end{pmatrix}.$$

Они совпали с независимо рассчитанными ожидаемыми значениями при одном и трёх участниках. При $K = 2$ все элементы результата эрозии равны 1, а наращивания — 9.

Дополнительно проверены $K = 0$, окно $1 \times 1$ при пяти проходах, запрос 20 потоков для трёх строк, а также отклонение параметров `-t 0`, `-k -1`, `-w 2`. Все девять перечисленных проверок завершились согласно ожиданиям. Журнал проверок:

```
PASS P=1 K=1 W=3
PASS P=3 K=1 W=3
PASS P=3 K=2 W=3
PASS P=3 K=0 W=3
PASS P=3 K=5 W=1
PASS P=20 K=1 W=3
PASS rejection -t 0
PASS rejection -k -1
PASS rejection -w 2
```

Фактический журнал запуска с тремя участниками: PID у участников одинаков, а TID различаются. Диапазоны строк не пересекаются. Порядок диагностических строк зависит от планировщика. Время короткого запуска с `-v` не используется для оценки ускорения.

## Подтверждение количества потоков средствами ОС

Для достаточно длительного запуска матрицы $1024 \times 1024$, $K = 100$, $W = 5$, $P = 4$ выполнена команда

```
ps -L -p "$pid" -o pid,tid,nlwp,comm
grep '^Threads:' /proc/"$pid"/status
```

Фактический вывод ps:

```
  PID TID NLWP COMMAND
  820 820 4 lab3
  820 821 4 lab3
  820 822 4 lab3
  820 823 4 lab3
```

Фрагмент /proc/PID/status:

```
Name: lab3
Pid: 820
Threads: 4
```

Обе проверки фиксируют четыре системных потока: главный и три дополнительных. PID и TID в этих журналах относятся только к данному запуску, а не являются постоянными значениями программы.

## Методика измерения производительности

Использованы матрицы $256 \times 256$ и $1024 \times 1024$, $K = 10$, окно $5 \times 5$ и $P \in \{1, 2, 4, 8\}$. Для каждой конфигурации выполнен один прогревочный запуск и пять измеряемых. В каждом повторе порядок значений $P$ перемешивался с фиксированным начальным состоянием генератора 12. В итоговую таблицу занесена медиана пяти измерений.

Матрицы являются специально созданными тестовыми входами, а не внешними экспериментальными данными: Python `random.Random(12+N)` генерировал целые значения от $-100000$ до $100000$, делённые на 100. Для всех чисел потоков проверено побайтовое совпадение двух выходных файлов с однопоточным результатом по SHA-256.

Время берётся из `TIME_SECONDS`. Оно включает создание дополнительных потоков, вычисления, ожидания на барьерах, присоединение потоков и освобождение их дескрипторов. Чтение исходного файла, предварительное выделение и копирование матриц, а также запись результатов за пределами этого интервала. Подробный вывод `-v` отключён.

Для фиксированных входных данных ускорение и эффективность определяются как

$$S_p = \frac{T_1}{T_p}, \qquad \eta_p = \frac{S_p}{p}.$$

Здесь $T_1$ и $T_p$ — медианы времени при одном и $p$ участниках. В таблице эффективность указана в процентах.

| Размер $N$ | Потоки $p$ | $T_p$, с | $S_p$ | $\eta_p$, % |
|---:|---:|---:|---:|---:|
| 256 | 1 | 0.018712 | 1.000 | 100.0 |
| 256 | 2 | 0.011057 | 1.692 | 84.6 |
| 256 | 4 | 0.011265 | 1.661 | 41.5 |
| 256 | 8 | 0.013531 | 1.383 | 17.3 |
| 1024 | 1 | 0.284596 | 1.000 | 100.0 |
| 1024 | 2 | 0.147220 | 1.933 | 96.7 |
| 1024 | 4 | 0.080648 | 3.529 | 88.2 |
| 1024 | 8 | 0.072673 | 3.916 | 49.0 |

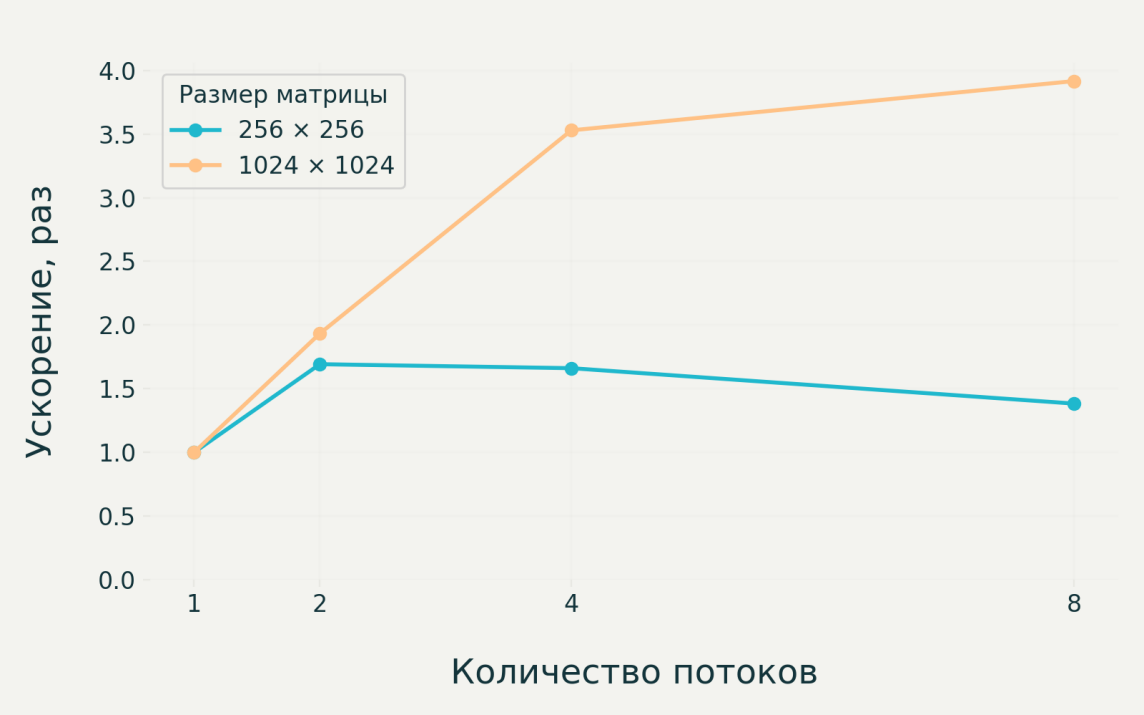



Рис. 1: Ускорение: $K = 10, W = 5$.

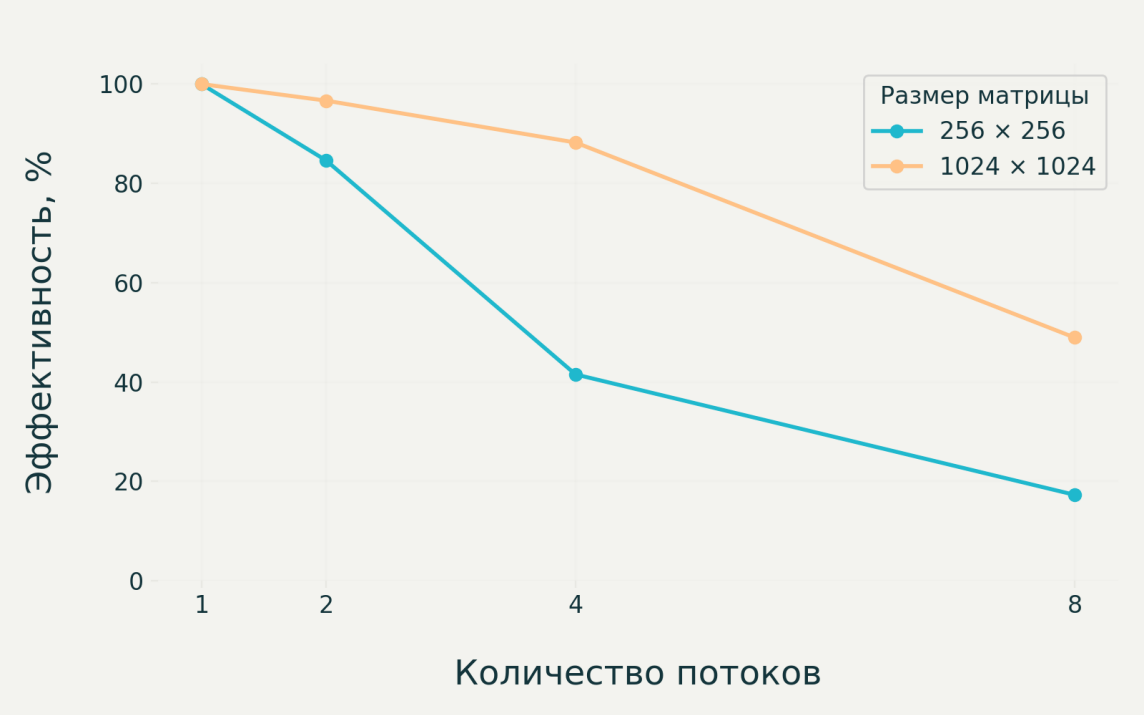



Рис. 2: Эффективность при изменении числа потоков.

## Объяснение результатов

Для матрицы $256 \times 256$ минимальная медиана времени получена при двух потоках: 0.011057 с против 0.018712 с при одном. Ускорение составляет 1.692, эффективность — 84.6%. При четырёх потоках время равно 0.011265 с, при восьми — 0.013531 с. Для небольшой задачи увеличение числа потоков не обеспечило дальнейшего уменьшения медианы времени.

Для матрицы $1024 \times 1024$ медиана снизилась с 0.284596 с при одном потоке до 0.147220 с при двух, 0.080648 с при четырёх и 0.072673 с при восьми. Максимальное наблюдаемое ускорение равно 3.916, эффективность восьми потоков — 49.0%. Эффективность двух и четырёх потоков составляет 96.7% и 88.2%.

Результаты отдельных повторов различаются. Например, для матрицы $256 \times 256$ при двух потоках время находится в диапазоне от 0.009704 до 0.029825 с. Поэтому в таблице и на графиках использованы медианы пяти повторов, а не минимальные значения. По этому эксперименту нельзя однозначно установить причины разброса; влияние фоновой нагрузки, планировщика и виртуализации отдельно не измерялось. Малое различие медиан двух и четырёх потоков на небольшой матрице не доказывает устойчивого превосходства.

Восемь конфигураций дали 40 измеряемых запусков после прогрева. Наличие 20 доступных логических CPU не означает линейного ускорения: время включает создание потоков, вычисления, барьеры и ожидание завершения. Полученные результаты относятся к указанной сборке и среде WSL2.

## Выводы

В работе рассмотрена многопоточная обработка матрицы вещественных чисел фильтрами эрозии и наращивания. Реализовано независимое $K$-кратное применение каждого фильтра с обрезанием окна на границе. Общее число участников ограничивается параметром запуска; главный поток также выполняет часть вычислений.

Работа с потоками вынесена в интерфейс `mt`, а Linux-операции создания, ожидания и синхронизации находятся в платформенной реализации. Разбиение по непересекающимся диапазонам строк позволяет выполнять вычисления без мьютекса на каждом элементе. Двойная буферизация и барьер между проходами обеспечивают чтение полностью рассчитанного предыдущего состояния матрицы.

Стартовая блокировка предотвращает зависание на барьере при частичном создании группы потоков. После ошибки уже созданные участники выходят до начала вычислений, а главный поток ожидает их завершения.

В ранее подготовленной контрольной Linux-сборке проверены эталонная матрица, граничные параметры и количество потоков средствами ОС. Измерения производительности текущей версии выполнены отдельно на компьютере студента в WSL2 с процессором Intel Core i7-13650HX.

Для матрицы $256 \times 256$ наименьшая медиана времени получена при двух потоках: ускорение 1.692 и эффективность 84.6%. Для матрицы $1024 \times 1024$ наименьшая медиана получена при восьми потоках: ускорение 3.916 и эффективность 49.0%. Увеличение объёма работы улучшило использование нескольких потоков. Рост числа потоков не обеспечил пропорционального ускорения.

В расчётах использованы медианы пяти повторов каждой конфигурации. Разброс отдельных запусков сохранён в исходных CSV; результаты характеризуют конкретную сборку и условия измерения, а не гарантированную производительность на другом компьютере.

## Исходная программа

Листинги подключаются непосредственно из рабочих исходников `../code/` и общей библиотеки `../../common/code/multithread/`. Отдельные копии исходного кода в отчёте не хранятся. Пути рассчитаны на компиляцию из каталога `lab3/report`. Журналы проверок и трассировки хранятся отдельно в `inc/`; они относятся к запускам, указанным в соответствующих разделах, и не обновляются автоматически при изменении исходников.

## Общие объявления

```
#pragma once
#include <cstddef>
#include <memory>
#include <string>

struct Options {
    std::string input;
    std::string prefix = "result";
    int threads = 1;
    int iterations = 1;
    int window = 3;
    bool verbose = false;
};

int parse_integer(const char* text, int minimum, const char* name);

[[noreturn]] void posix_error(const char* operation, int code);
[[noreturn]] void fatal_posix(const char* operation, int code) noexcept;

void write_stdout(const char* text, std::size_t count) noexcept;

double monotonic_seconds();
long linux_tid() noexcept;

struct Matrix {
    int rows = 0;
    int cols = 0;
    std::unique_ptr<double[]> values;

    Matrix() = default;
    Matrix(int height, int width);
    Matrix(Matrix&&) noexcept = default;

    Matrix& operator=(Matrix&&) noexcept = default;
    Matrix(const Matrix&) = delete;
    Matrix& operator=(const Matrix&) = delete;
    std::size_t count() const noexcept;
};

Matrix read_matrix(const std::string& path);
void write_matrix(const std::string& path, const Matrix& matrix);
Matrix copy_matrix(const Matrix& matrix);
```

Листинг 1: Структуры данных и общие функции

## Интерфейс фильтров

```hpp
#pragma once
#include "common.hpp"

struct Result {
    Matrix erosion;
    Matrix dilation;
    double seconds = 0;
    int threads = 1;
};

Result run_filters(Matrix input, const Options& options);
```

Листинг 2: Результат и интерфейс обработки матрицы

## Главная функция

```cpp
#include "filters.hpp"
#include <cerrno>
#include <csignal>
#include <cstdio>
#include <cstdlib>
#include <exception>
#include <getopt.h>
#include <new>
#include <stdexcept>
#include <unistd.h>
#include <utility>

namespace {
const char usage[] =
    "Usage: lab3 -i FILE [-t P] [-k K] [-w W] [-o PREFIX] [-v]\n"
    " -t P maximum threads INCLUDING main (default 1)\n"
    " -k K repetitions of each independent filter (default 1)\n"
    " -w W positive odd square window size (default 3)\n"
    " -o S output prefix (default result)\n"
    " -v PID/TID and row range for each participant\n";

void ignore_sigpipe() {
    struct sigaction action{};
    action.sa_handler = SIG_IGN;
    if (sigemptyset(&action.sa_mask) != 0) posix_error("sigemptyset", errno);
    if (sigaction(SIGPIPE, &action, nullptr) != 0) posix_error("sigaction", errno);
}

Options parse_options(int argc, char** argv) {
    Options o;
    opterr = 0;
    int flag;
    while ((flag = getopt(argc, argv, "i:t:k:w:o:vh")) != -1) {
        switch (flag) {
        case 'i': o.input = optarg; break;
        case 't': o.threads = parse_integer(optarg, 1, "thread count"); break;
        case 'k': o.iterations = parse_integer(optarg, 0, "iteration count"); break;
        case 'w': o.window = parse_integer(optarg, 1, "window size"); break;
        case 'o': o.prefix = optarg; break;
```

```
        case 'v': o.verbose = true; break;
        case 'h': write_stdout(usage, sizeof usage - 1); std::exit(EXIT_SUCCESS);
        default: throw std::runtime_error("Unknown option or missing option value");
        }
    }
    if (o.input.empty() || o.prefix.empty() || optind != argc)
        throw std::runtime_error("Use -i FILE and a nonempty output prefix; no
            positional arguments");
    if (o.window % 2 == 0) throw std::runtime_error("Window must be odd");
    return o;
}
}

int main(int argc, char** argv) {
    try {
        ignore_sigpipe();
        const Options options = parse_options(argc, argv);
        Matrix input = read_matrix(options.input);

        const int actual = options.iterations == 0 ? 1 :
            (options.threads < input.rows ? options.threads : input.rows);

        char line[256];

        int n = std::snprintf(line, sizeof line,
            "PID=%ld TID=%ld ROWS=%d COLS=%d THREADS=%d K=%d W=%d\n",
            static_cast<long>(::getpid()), linux_tid(), input.rows, input.cols,
            actual, options.iterations, options.window);

        if (n < 0 || static_cast<std::size_t>(n) >= sizeof line)
            throw std::runtime_error("Cannot format metadata");

        write_stdout(line, static_cast<std::size_t>(n));
        Result result = run_filters(std::move(input), options);

        write_matrix(options.prefix + "_erosion.txt", result.erosion);
        write_matrix(options.prefix + "_dilation.txt", result.dilation);

        n = std::snprintf(line, sizeof line, "TIME_SECONDS=%.9f THREADS=%d\n",
                          result.seconds, result.threads);

        if (n < 0 || static_cast<std::size_t>(n) >= sizeof line)
            throw std::runtime_error("Cannot format timing");

        write_stdout(line, static_cast<std::size_t>(n));
        return EXIT_SUCCESS;
    } catch (const std::bad_alloc&) {
        const char message[] = "ERROR: Not enough memory (allocation failed)\n";
        if (::write(STDERR_FILENO, message, sizeof message - 1) < 0)
            return EXIT_FAILURE;
    } catch (const std::exception& e) {
        if (std::fprintf(stderr, "ERROR: %s\n", e.what()) < 0)
            return EXIT_FAILURE;
        if (std::fflush(stderr) != 0) return EXIT_FAILURE;
    }
    return EXIT_FAILURE;
```

```
}
```

Листинг 3: Аргументы запуска и организация обработки

## Вспомогательные функции

```
#include "common.hpp"
#include <cerrno>
#include <climits>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <stdexcept>
#include <sys/syscall.h>
#include <time.h>
#include <unistd.h>

int parse_integer(const char* text, int minimum, const char* name) {
    char* end = nullptr;
    errno = 0;
    const long value = std::strtol(text, &end, 10);

    if (errno != 0 || text == end || *end != '\0' ||
        value < minimum || value > INT_MAX) {
        throw std::runtime_error(std::string("Invalid ") + name + ": " + text);
    }

    return static_cast<int>(value);
}

[[noreturn]] void posix_error(const char* operation, int code) {
    throw std::runtime_error(std::string(operation) + ": " + std::strerror(code));
}

[[noreturn]] void fatal_posix(const char* operation, int code) noexcept {
    char buffer[512];
    int count = std::snprintf(buffer, sizeof buffer, "FATAL: %s: error=%d\n",
        operation, code);

    if (count > 0 && static_cast<std::size_t>(count) < sizeof buffer) {
        std::size_t done = 0;
        while (done < static_cast<std::size_t>(count)) {
            const ssize_t n = ::write(STDERR_FILENO, buffer + done,
                                      static_cast<std::size_t>(count) - done);
            if (n < 0 && errno == EINTR) continue;
            if (n <= 0) break;
            done += static_cast<std::size_t>(n);
        }
    }
    std::_Exit(EXIT_FAILURE);
}

void write_stdout(const char* text, std::size_t count) noexcept {
    std::size_t done = 0;
    while (done < count) {
        const ssize_t n = ::write(STDOUT_FILENO, text + done, count - done);
        if (n < 0 && errno == EINTR) continue;
```

```
        if (n <= 0) fatal_posix("write(stdout)", n < 0 ? errno : EIO);
        done += static_cast<std::size_t>(n);
    }
}

double monotonic_seconds() {
    timespec ts{};
    if (::clock_gettime(CLOCK_MONOTONIC, &ts) != 0)
        posix_error("clock_gettime", errno);
    return static_cast<double>(ts.tv_sec) + static_cast<double>(ts.tv_nsec) / 1e9;
}

long linux_tid() noexcept {
    const long tid = ::syscall(SYS_gettid);
    if (tid == -1) fatal_posix("gettid", errno);
    return tid;
}
```

Листинг 4: Диагностика и измерение времени

## Матрицы и файловый ввод-вывод

```
#include "common.hpp"
#include <cerrno>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <limits>
#include <stdexcept>

namespace {
class File {
    FILE* file_;
public:
    File(const std::string& path, const char* mode) : file_(std::fopen(path.c_str(),
        mode)) {
        if (!file_) posix_error("fopen", errno);
    }
    ~File() {
        if (file_ && std::fclose(file_) != 0) fatal_posix("fclose(cleanup)", errno);
    }
    FILE* get() const noexcept { return file_; }
    void close() {
        FILE* temporary = file_;
        file_ = nullptr; // fclose consumes the stream even on failure.
        if (std::fclose(temporary) != 0) posix_error("fclose", errno);
    }
};

bool token(FILE* file, char (&buffer)[256]) {
    const int result = std::fscanf(file, "%255s", buffer);
    if (result == EOF) {
        if (std::ferror(file)) posix_error("fscanf", errno ? errno : EIO);
        return false;
    }
    if (result != 1) throw std::runtime_error("Input token cannot be read");
```

```cpp
    if (std::strlen(buffer) == 255) throw std::runtime_error("Input token too long");
    return true;
}
}

Matrix::Matrix(int height, int width) : rows(height), cols(width) {
    if (rows <= 0 || cols <= 0) throw std::runtime_error("Invalid matrix dimensions");
    const auto maximum = std::numeric_limits<std::size_t>::max();
    if (static_cast<std::size_t>(rows) > maximum / static_cast<std::size_t>(cols) ||
        count() > maximum / sizeof(double))
        throw std::runtime_error("Matrix size overflow");
    values.reset(new double[count()]);
}

std::size_t Matrix::count() const noexcept {
    return static_cast<std::size_t>(rows) * static_cast<std::size_t>(cols);
}

Matrix copy_matrix(const Matrix& matrix) {
    Matrix result(matrix.rows, matrix.cols);
    std::memcpy(result.values.get(), matrix.values.get(), matrix.count() * sizeof(
        double));
    return result;
}

Matrix read_matrix(const std::string& path) {
    File file(path, "r");
    char buffer[256];
    if (!token(file.get(), buffer)) throw std::runtime_error("Missing row count");
    const int rows = parse_integer(buffer, 1, "row count");
    if (!token(file.get(), buffer)) throw std::runtime_error("Missing column count");
    const int cols = parse_integer(buffer, 1, "column count");
    Matrix matrix(rows, cols);
    for (std::size_t i = 0; i < matrix.count(); ++i) {
        if (!token(file.get(), buffer)) throw std::runtime_error("Too few matrix values
            ");
        char* end = nullptr;
        errno = 0;
        const double value = std::strtod(buffer, &end);
        if (errno != 0 || end == buffer || *end != '\0' || !std::isfinite(value))
            throw std::runtime_error("Invalid finite double in matrix");
        matrix.values[i] = value;
    }
    if (token(file.get(), buffer)) throw std::runtime_error("Extra data after matrix")
        ;
    file.close();
    return matrix;
}

void write_matrix(const std::string& path, const Matrix& matrix) {
    File file(path, "w");
    if (std::fprintf(file.get(), "%d %d\n", matrix.rows, matrix.cols) < 0)
        posix_error("fprintf(header)", errno ? errno : EIO);
    for (int i = 0; i < matrix.rows; ++i) {
        for (int j = 0; j < matrix.cols; ++j) {
            const std::size_t index = static_cast<std::size_t>(i) * matrix.cols + j;
            if (std::fprintf(file.get(), "%.17g%c", matrix.values[index],
                             j + 1 == matrix.cols ? '\n' : ' ') < 0)
```

```
                posix_error("fprintf(matrix)", errno ? errno : EIO);
            }
        }
        file.close();
}
```

Листинг 5: Хранение и проверка элементов матрицы

## Многопоточные фильтры

```
#include "filters.hpp"
#include <cerrno>
#include <cstdio>
#include <limits>
#include <stdexcept>
#include "multithread/mt.h"
#include "multithread/mt_sync.h"
#include <exception>
#include <unistd.h>
#include <utility>

namespace {
class Mutex {
    mt::MutexHandle handle_ = mt::MutexCreate();
public:
    Mutex() = default;
    Mutex(const Mutex&) = delete;
    Mutex& operator=(const Mutex&) = delete;
    ~Mutex() { mt::MutexDestroy(handle_); }
    void lock() noexcept { mt::MutexLock(handle_); }
    void unlock() noexcept { mt::MutexUnlock(handle_); }
};

class Barrier {
    mt::BarrierHandle handle_;
public:
    explicit Barrier(unsigned participants) : handle_(mt::BarrierCreate(participants))
        {}
    Barrier(const Barrier&) = delete;
    Barrier& operator=(const Barrier&) = delete;
    ~Barrier() { mt::BarrierDestroy(handle_); }
    void wait() noexcept { mt::BarrierWait(handle_); }
};

struct Context {
    Matrix erosion[2];
    Matrix dilation[2];
    int iterations;
    int radius;
    bool verbose;
    bool abort_start = false;
    Barrier barrier;
    Mutex gate;
    Mutex log;
    Context(int k, int r, int p, bool v)
        : iterations(k), radius(r), verbose(v), barrier(static_cast<unsigned>(p)) {}
};
```

```
47
48  struct Task {
49      Context* context;
50      int id;
51      int first;
52      int last;
53  };
54
55  void log_worker(Context& context, const Task& task, const char* event) noexcept {
56      char line[256];
57      const int n = std::snprintf(line, sizeof line,
58          "%s PID=%ld TID=%ld worker=%d rows=[%d,%d)\n", event,
59          static_cast<long>(::getpid()), linux_tid(), task.id, task.first, task.last);
60      if (n < 0 || static_cast<std::size_t>(n) >= sizeof line)
61          fatal_posix("snprintf(log)", EIO);
62      context.log.lock();
63      write_stdout(line, static_cast<std::size_t>(n));
64      context.log.unlock();
65  }
66
67  void worker(void* opaque) noexcept {
68      const Task& task = *static_cast<Task*>(opaque);
69      Context& c = *task.context;
70      c.gate.lock();
71      const bool abort = c.abort_start;
72      c.gate.unlock();
73      if (abort) return;
74
75      if (c.verbose) log_worker(c, task, "START");
76      const int rows = c.erosion[0].rows;
77      const int cols = c.erosion[0].cols;
78      for (int step = 0; step < c.iterations; ++step) {
79          const int src = step % 2;
80          const int dst = 1 - src;
81          for (int i = task.first; i < task.last; ++i) {
82              const int top = i > c.radius ? i - c.radius : 0;
83              const int bottom = c.radius < rows - 1 - i ? i + c.radius : rows - 1;
84              for (int j = 0; j < cols; ++j) {
85                  const int left = j > c.radius ? j - c.radius : 0;
86                  const int right = c.radius < cols - 1 - j ? j + c.radius : cols - 1;
87                  double low = std::numeric_limits<double>::infinity();
88                  double high = -std::numeric_limits<double>::infinity();
89                  for (int x = top; x <= bottom; ++x) {
90                      const std::size_t base = static_cast<std::size_t>(x) * cols;
91                      for (int y = left; y <= right; ++y) {
92                          const double e = c.erosion[src].values[base + y];
93                          const double d = c.dilation[src].values[base + y];
94                          if (e < low) low = e;
95                          if (d > high) high = d;
96                      }
97                  }
98                  const std::size_t index = static_cast<std::size_t>(i) * cols + j;
99                  c.erosion[dst].values[index] = low;
100                 c.dilation[dst].values[index] = high;
101             }
102         }
103         c.barrier.wait();
104     }
```

```
    if (c.verbose) log_worker(c, task, "DONE");
}
}

Result run_filters(Matrix input, const Options& options) {
    if (options.iterations == 0) {
        Matrix dilation = copy_matrix(input);
        return {std::move(input), std::move(dilation), 0.0, 1};
    }
    const int p = options.threads < input.rows ? options.threads : input.rows;
    Context c(options.iterations, options.window / 2, p, options.verbose);
    c.erosion[0] = std::move(input);
    c.erosion[1] = Matrix(c.erosion[0].rows, c.erosion[0].cols);
    c.dilation[0] = copy_matrix(c.erosion[0]);
    c.dilation[1] = Matrix(c.erosion[0].rows, c.erosion[0].cols);
    std::unique_ptr<Task[]> tasks(new Task[p]);
    std::unique_ptr<mt::ThreadHandle[]> handles(new mt::ThreadHandle[p - 1]{});
    for (int i = 0; i < p; ++i) {
        tasks[i] = {&c, i,
            static_cast<int>(static_cast<long long>(c.erosion[0].rows) * i / p),
            static_cast<int>(static_cast<long long>(c.erosion[0].rows) * (i + 1) / p)};
    }

    const double start = monotonic_seconds();
    c.gate.lock();
    int created = 0;
    std::exception_ptr create_error;
    try {
        for (int i = 1; i < p; ++i) {
            handles[created] = mt::CreateThread(&worker, &tasks[i],
                                                "lab3-" + std::to_string(i));
            if (!handles[created])
                throw std::runtime_error("mt::CreateThread returned a null handle");
            ++created;
        }
    } catch (...) {
        // No worker may enter the barrier unless all participants were created.
        c.abort_start = true;
        create_error = std::current_exception();
    }
    c.gate.unlock();
    if (!create_error) worker(&tasks[0]);
    for (int i = 0; i < created; ++i) {
        try {
            mt::ThreadJoin(handles[i]);
            mt::DestroyThreadHandle(handles[i]);
            handles[i] = nullptr;
        } catch (...) {
            // Context/task storage must not be freed while a worker can access it.
            fatal_posix("mt::ThreadJoin/DestroyThreadHandle", EIO);
        }
    }
    if (create_error) std::rethrow_exception(create_error);
    const double seconds = monotonic_seconds() - start;

    const int index = options.iterations % 2;
    return {std::move(c.erosion[index]), std::move(c.dilation[index]), seconds, p};
```

```
}
```

Листинг 6: Фильтрация через интерфейс mt

## Интерфейс библиотеки потоков

```
#pragma once
#include <cstdint>

#include "common/comm.h"
#include <functional>

namespace mt {
struct ThreadHandleDummy { };
typedef ThreadHandleDummy* ThreadHandle;

using FuncNoArgs = std::function<void()>;
typedef void (*ThreadFunc)(void*);

using ThreadID = uint64_t;
using ThreadNameType = std::string;

// Create/Terminate functions
ThreadHandle CreateThread(ThreadFunc func, void* param = nullptr, const ThreadNameType
    & name = "", int stackSize = -1);
ThreadHandle CreateThread(FuncNoArgs func, const ThreadNameType& name = "", int
    stackSize = -1);
int ThreadTerminate(ThreadHandle handle);
void DestroyThreadHandle(ThreadHandle handle);

// Join/Detach
void ThreadJoin(ThreadHandle handle, uint32_t msTimeout = -1);
void ThreadDetach(ThreadHandle handle);
};
```

Листинг 7: Интерфейс библиотеки mt

## Платформенная реализация потоков

```
#ifndef _GNU_SOURCE
#define _GNU_SOURCE
#endif
#include <cstdint>
#include "mt.h"
#include <cerrno>
#include <cstring>
#include <memory>
#include <pthread.h>
#include <stdexcept>
#include <time.h>
#include <utility>

namespace {
struct NativeThread {
    pthread_t thread{};
    bool joined = false;
```

```
};
}

namespace {
[[noreturn]] void fail(const char* operation, int error) {
    throw std::runtime_error(std::string(operation) + ": " + std::strerror(error));
}

struct StartArgs {
    mt::ThreadFunc func;
    void* parameter;
    std::string name;
};

void* start_thread(void* opaque) {
    std::unique_ptr<StartArgs> args(static_cast<StartArgs*>(opaque));
    // Linux thread names are optional diagnostics, limited to 15 visible bytes.
    if (!args->name.empty())
        (void)pthread_setname_np(pthread_self(), args->name.c_str());
    const auto function = args->func;
    void* parameter = args->parameter;
    args.reset();
    function(parameter);
    return nullptr;
}

void start_function(void* opaque) {
    std::unique_ptr<mt::FuncNoArgs> function(static_cast<mt::FuncNoArgs*>(opaque));
    (*function)();
}
}

mt::ThreadHandle mt::CreateThread(ThreadFunc function, void* parameter,
                                  const ThreadNameType& name, int stackSize) {
    if (!function) throw std::invalid_argument("CreateThread: null entry point");
    if (stackSize < -1) throw std::invalid_argument("CreateThread: invalid stack size"
        );
    auto handle = std::make_unique<NativeThread>();
    auto args = std::make_unique<StartArgs>(StartArgs{function, parameter, name.substr
        (0, 15)});
    pthread_attr_t attributes;
    int error = pthread_attr_init(&attributes);
    if (error != 0) fail("pthread_attr_init", error);
    if (stackSize > 0) {
        error = pthread_attr_setstacksize(&attributes, static_cast<std::size_t>(
            stackSize));
        if (error != 0) {
            (void)pthread_attr_destroy(&attributes);
            fail("pthread_attr_setstacksize", error);
        }
    }
    // Release before starting: the new thread can finish before pthread_create
        returns.
    StartArgs* raw = args.release();
    error = pthread_create(&handle->thread, &attributes, start_thread, raw);
    (void)pthread_attr_destroy(&attributes);
    if (error != 0) {
        args.reset(raw);
```

```
72          fail("pthread_create", error);
73      }
74      return reinterpret_cast<ThreadHandle>(handle.release());
75  }
76
77  mt::ThreadHandle mt::CreateThread(FuncNoArgs function, const ThreadNameType& name, int
        stackSize) {
78      auto argument = std::make_unique<FuncNoArgs>(std::move(function));
79      FuncNoArgs* raw = argument.release();
80      try {
81          return CreateThread(&start_function, raw, name, stackSize);
82      } catch (...) {
83          argument.reset(raw);
84          throw;
85      }
86  }
87
88  void mt::ThreadJoin(ThreadHandle opaque, uint32_t milliseconds) {
89      auto* handle = reinterpret_cast<NativeThread*>(opaque);
90      if (!handle) throw std::invalid_argument("ThreadJoin: null handle");
91      if (handle->joined) return;
92      int error;
93      if (milliseconds == static_cast<uint32_t>(-1)) {
94          error = pthread_join(handle->thread, nullptr);
95      } else {
96          timespec deadline{};
97          if (clock_gettime(CLOCK_REALTIME, &deadline) != 0) fail("clock_gettime", errno)
                ;
98          deadline.tv_sec += milliseconds / 1000;
99          deadline.tv_nsec += static_cast<long>(milliseconds % 1000) * 1000000L;
100         if (deadline.tv_nsec >= 1000000000L) {
101             ++deadline.tv_sec;
102             deadline.tv_nsec -= 1000000000L;
103         }
104         error = pthread_timedjoin_np(handle->thread, nullptr, &deadline);
105     }
106     if (error != 0) fail("pthread_join", error);
107     handle->joined = true;
108 }
109
110 void mt::DestroyThreadHandle(ThreadHandle opaque) {
111     auto* handle = reinterpret_cast<NativeThread*>(opaque);
112     if (!handle) return;
113     if (!handle->joined) {
114         const int error = pthread_detach(handle->thread);
115         if (error != 0) fail("pthread_detach", error);
116     }
117     delete handle;
118 }
119
120 void mt::ThreadDetach(ThreadHandle handle) {
121     DestroyThreadHandle(handle);
122 }
123
124 int mt::ThreadTerminate(ThreadHandle opaque) {
125     auto* handle = reinterpret_cast<NativeThread*>(opaque);
126     // Compatibility with mt.h only. Lab3 never forcibly cancels workers.
127     if (!handle) return EINVAL;
```

```
    if (handle->joined) {
        delete handle;
        return 0;
    }
    int error = pthread_cancel(handle->thread);
    if (error != 0) return error;
    error = pthread_join(handle->thread, nullptr);
    if (error != 0) return error;
    delete handle;
    return 0;
}
```

Листинг 8: Реализация через POSIX Threads в Linux

## Интерфейс синхронизации

```
#pragma once

// Additions for lab3; existing thread API signatures are preserved.
#include <cstdint>
#include "mt.h"

namespace mt {
struct MutexHandleDummy;
using MutexHandle = MutexHandleDummy*;
struct BarrierHandleDummy;
using BarrierHandle = BarrierHandleDummy*;

// Creation throws std::runtime_error on an OS error.
// Operations/destruction are noexcept: unrecoverable errors terminate the process.
MutexHandle MutexCreate();
void MutexDestroy(MutexHandle handle) noexcept;
void MutexLock(MutexHandle handle) noexcept;
void MutexUnlock(MutexHandle handle) noexcept;

BarrierHandle BarrierCreate(unsigned participants);
void BarrierDestroy(BarrierHandle handle) noexcept;
void BarrierWait(BarrierHandle handle) noexcept;
}
```

Листинг 9: Мьютекс и барьер

## Реализация синхронизации

```
#ifndef _GNU_SOURCE
#define _GNU_SOURCE
#endif
#include "mt_sync.h"
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <memory>
#include <pthread.h>
#include <stdexcept>

namespace mt {
```

```
13 struct MutexHandleDummy { pthread_mutex_t value; };
14 struct BarrierHandleDummy { pthread_barrier_t value; };
15 }
16
17 namespace {
18 [[noreturn]] void fail(const char* operation, int error) {
19     throw std::runtime_error(std::string(operation) + ": " + std::strerror(error));
20 }
21 [[noreturn]] void fatal(const char* operation, int error) noexcept {
22     (void)std::fprintf(stderr, "FATAL: %s: error=%d\n", operation, error);
23     (void)std::fflush(stderr);
24     std::_Exit(EXIT_FAILURE);
25 }
26 }
27
28 mt::MutexHandle mt::MutexCreate() {
29     auto mutex = std::make_unique<MutexHandleDummy>();
30     const int error = pthread_mutex_init(&mutex->value, nullptr);
31     if (error != 0) fail("pthread_mutex_init", error);
32     return mutex.release();
33 }
34 void mt::MutexDestroy(MutexHandle handle) noexcept {
35     if (!handle) return;
36     const int error = pthread_mutex_destroy(&handle->value);
37     if (error != 0) fatal("pthread_mutex_destroy", error);
38     delete handle;
39 }
40 void mt::MutexLock(MutexHandle handle) noexcept {
41     const int error = pthread_mutex_lock(&handle->value);
42     if (error != 0) fatal("pthread_mutex_lock", error);
43 }
44 void mt::MutexUnlock(MutexHandle handle) noexcept {
45     const int error = pthread_mutex_unlock(&handle->value);
46     if (error != 0) fatal("pthread_mutex_unlock", error);
47 }
48
49 mt::BarrierHandle mt::BarrierCreate(unsigned participants) {
50     if (participants == 0) throw std::invalid_argument("BarrierCreate: zero
        participants");
51     auto barrier = std::make_unique<BarrierHandleDummy>();
52     const int error = pthread_barrier_init(&barrier->value, nullptr, participants);
53     if (error != 0) fail("pthread_barrier_init", error);
54     return barrier.release();
55 }
56 void mt::BarrierDestroy(BarrierHandle handle) noexcept {
57     if (!handle) return;
58     const int error = pthread_barrier_destroy(&handle->value);
59     if (error != 0) fatal("pthread_barrier_destroy", error);
60     delete handle;
61 }
62 void mt::BarrierWait(BarrierHandle handle) noexcept {
63     const int error = pthread_barrier_wait(&handle->value);
64     if (error != 0 && error != PTHREAD_BARRIER_SERIAL_THREAD)
65         fatal("pthread_barrier_wait", error);
66 }
```

Листинг 10: Операции POSIX-синхронизации

## Трассировка системных вызовов

Контрольный запуск трассировался командой следующего вида:

```
strace -f -tt -o inc/strace.txt \
  -e trace=clone,clone3,futex,gettid,write \
  ./lab3 -i small.txt -t 3 -k 2 -w 3 -o trace_result -v
```

Полный фактический журнал приведён ниже. В нём видны два успешных `clone3` для двух дополнительных потоков, вызовы `gettid` и операции `futex`. Вместе с главным потоком это соответствует $P = 3$. Операции `futex` могут относиться как к примитивам синхронизации, так и к ожиданию завершения; не каждому вызову `pthread_*` обязательно соответствует отдельный системный вызов. См. также clone(2), futex(2) и strace(1).

```
898 07:58:46.249299 futex(0x7efe9029c83c, FUTEX_WAKE_PRIVATE, 2147483647) = 0
898 07:58:46.249607 gettid() = 898
898 07:58:46.249654 write(1, "PID=898 TID=898 ROWS=3 COLS=3 TH"..., 48) = 48
898 07:58:46.249824 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
    CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7efe8fbffce8, parent_tid=0
    x7efe8fbff990, exit_signal=0, stack=0x7efe8f3ff000, stack_size=0x7fff40, tls=0x7efe8fbff6c0} => {
    parent_tid=[899]}, 88) = 899
898 07:58:46.249995 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
    CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7efe8f3fece8, parent_tid=0
    x7efe8f3fe990, exit_signal=0, stack=0x7efe8ebfe000, stack_size=0x7fff40, tls=0x7efe8f3fe6c0} => {
    parent_tid=[900]}, 88) = 900
898 07:58:46.250103 gettid( <unfinished ...>
899 07:58:46.250114 gettid( <unfinished ...>
898 07:58:46.250121 <... gettid resumed>) = 898
899 07:58:46.250130 <... gettid resumed>) = 899
898 07:58:46.250156 write(1, "START PID=898 TID=898 worker=0 r"..., 42 <unfinished ...>
899 07:58:46.250170 futex(0x7ffcc578df98, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
898 07:58:46.250182 <... write resumed>) = 42
898 07:58:46.250196 futex(0x7ffcc578df98, FUTEX_WAKE_PRIVATE, 1) = 1
899 07:58:46.250216 <... futex resumed>) = 0
898 07:58:46.250223 futex(0x7ffcc578df54, FUTEX_WAIT_PRIVATE, 0, NULL <unfinished ...>
899 07:58:46.250232 write(1, "START PID=898 TID=899 worker=1 r"..., 42) = 42
899 07:58:46.250260 futex(0x7ffcc578df98, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>
900 07:58:46.250267 gettid( <unfinished ...>
899 07:58:46.250273 <... futex resumed>) = 0
900 07:58:46.250279 <... gettid resumed>) = 900
899 07:58:46.250286 futex(0x7ffcc578df54, FUTEX_WAIT_PRIVATE, 0, NULL <unfinished ...>
900 07:58:46.250311 write(1, "START PID=898 TID=900 worker=2 r"..., 42) = 42
900 07:58:46.250336 futex(0x7ffcc578df54, FUTEX_WAKE_PRIVATE, 2147483647) = 2
899 07:58:46.250355 <... futex resumed>) = 0
900 07:58:46.250362 futex(0x7ffcc578df54, FUTEX_WAIT_PRIVATE, 3, NULL <unfinished ...>
899 07:58:46.250368 futex(0x7ffcc578df54, FUTEX_WAIT_PRIVATE, 3, NULL <unfinished ...>
898 07:58:46.250384 <... futex resumed>) = 0
898 07:58:46.250401 futex(0x7ffcc578df54, FUTEX_WAKE_PRIVATE, 2147483647) = 2
899 07:58:46.250421 <... futex resumed>) = 0
898 07:58:46.250427 gettid( <unfinished ...>
899 07:58:46.250434 gettid( <unfinished ...>
898 07:58:46.250440 <... gettid resumed>) = 898
899 07:58:46.250447 <... gettid resumed>) = 899
898 07:58:46.250477 write(1, "DONE PID=898 TID=898 worker=0 ro"..., 41 <unfinished ...>
900 07:58:46.250499 <... futex resumed>) = 0
899 07:58:46.250508 futex(0x7ffcc578df98, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
898 07:58:46.250515 <... write resumed>) = 41
900 07:58:46.250521 gettid( <unfinished ...>
898 07:58:46.250527 futex(0x7ffcc578df98, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>
900 07:58:46.250534 <... gettid resumed>) = 900
898 07:58:46.250541 <... futex resumed>) = 1
899 07:58:46.250547 <... futex resumed>) = 0
898 07:58:46.250553 futex(0x7efe8fbffce8, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 2, NULL,
    FUTEX_BITSET_MATCH_ANY <unfinished ...>
899 07:58:46.250563 write(1, "DONE PID=898 TID=899 worker=1 ro"..., 41 <unfinished ...>
900 07:58:46.250582 futex(0x7ffcc578df98, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
899 07:58:46.250588 <... write resumed>) = 41
899 07:58:46.250600 futex(0x7ffcc578df98, FUTEX_WAKE_PRIVATE, 1) = 1
900 07:58:46.250620 <... futex resumed>) = 0
900 07:58:46.250632 write(1, "DONE PID=898 TID=900 worker=2 ro"..., 41) = 41
900 07:58:46.250665 futex(0x7ffcc578df98, FUTEX_WAKE_PRIVATE, 1) = 0
```

```
899 07:58:46.250701 +++ exited with 0 +++
898 07:58:46.250705 <... futex resumed>) = 0
898 07:58:46.250720 futex(0x7efe8f3fece8, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 1, NULL,
     FUTEX_BITSET_MATCH_ANY) = 0
900 07:58:46.250772 +++ exited with 0 +++
898 07:58:46.250854 write(3, "3 3\n1 1 1\n1 1 1\n1 1 1\n", 22) = 22
898 07:58:46.250963 write(3, "3 3\n9 9 9\n9 9 9\n9 9 9\n", 22) = 22
898 07:58:46.251010 write(1, "TIME_SECONDS=0.001062679 THREADS"..., 35) = 35
898 07:58:46.251205 +++ exited with 0 +++
```

Листинг 11: Реальная трассировка контрольного запуска